Project – Low Level Design

On

Multimodal Manufacturing Creator

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Om Tiwari	EN22CS301669
02	Paridhi Shirwalkar	EN22CS301684
03	Nitesh Chourasiya	EN22CS301660
04	Mradul Jain	EN22CS301616

Group Name: 01D6

Project Number: GAI-12

Industry Mentor Name: Prof. Akshay Saxena

University Mentor Name:

Academic Year: 2025-2026

Table of Contents

1. Introduction.

    1.1. Scope of the document.
    1.2. Intended Audience
    1.3. System overview.

2. System Design.

    2.1. Application Design
    2.2. Process Flow.
    2.3. Information Flow.
    2.4. Components Design
    2.5. Key Design Considerations
    2.6. API Catalogue.

3. References

4. GitHub

1. Introduction

The Multimodal Manufacturing Creator is a Generative AI application designed to bridge the gap between textual descriptions and visual representations in the manufacturing domain. By integrating two distinct AI model types a Large Language Model (LLM) for text generation and a diffusion-based image generation model for visual synthesis the system enables users to explore, understand, and communicate complex manufacturing concepts through both narrative and imagery.

This document describes the system design of the application, covering its architecture, process flow, information flow, component design, and API integrations.

1.1 Scope of the Document

This Software Design Document (SDD) covers the design and architecture of the Multimodal Manufacturing Creator application. It describes the application layer, the integration of external AI APIs, data flow between components, and the design decisions that govern the system. It does not cover deployment infrastructure, CI/CD pipelines, or hardware specifications.

1.2 Intended Audience

This document is intended for the following stakeholders:

- Datagami course evaluators and industry mentors reviewing the project
- Developers who may extend or maintain the system in future
- Technical reviewers assessing the GenAI implementation and API integration
- Academic peers seeking to understand multimodal AI application design

1.3 System Overview

The Multimodal Manufacturing Creator accepts a single natural language prompt describing a manufacturing concept from the user. This prompt is simultaneously fed to two AI models: Google Gemini 2.5 Flash, a Large Language Model that generates a structured five-section technical narrative, and the Flux diffusion model served via Pollinations.ai, which generates a high-fidelity visual illustration of the concept.

The frontend is built using Streamlit, a Python-based web application framework. The system requires no database and maintains no persistent state — all processing is stateless and session-scoped. The user receives both outputs rendered side by side within a single interface.

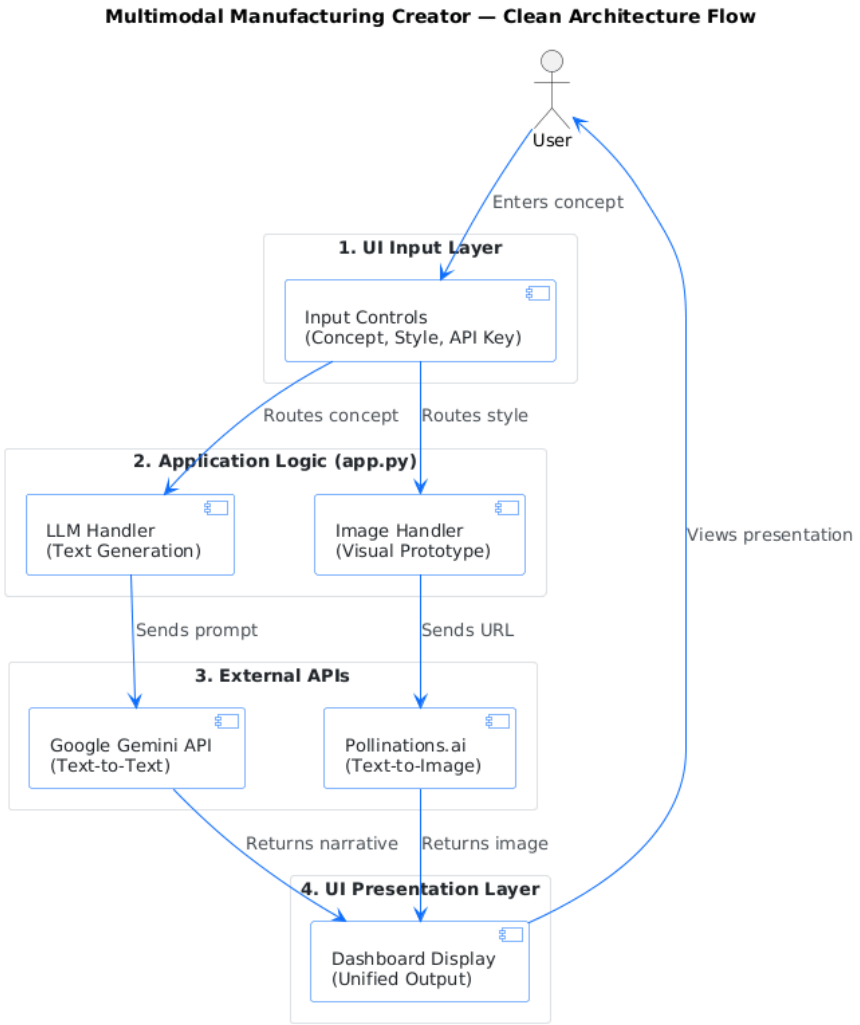
2. System Design

2.1 Application Design

The application follows a single-tier frontend architecture where the Streamlit application acts as both the user interface and the orchestration layer. There is no dedicated backend server all API calls are made directly from the Streamlit session process. The design is intentionally minimal to keep the focus on the multimodal AI workflow rather than infrastructure complexity.

The application is structured into four logical layers:

- **Presentation Layer:** Streamlit UI components for user input, output rendering, and workflow visualization
- **Orchestration Layer:** Python logic that coordinates parallel calls to both AI APIs from a single user prompt
- **Text Generation Layer:** Google Gemini 2.5 Flash API called via the google-generativeai Python SDK
- **Image Generation Layer:** Pollinations.ai Flux model called via an HTTP GET request constructed from the encoded prompt



2.2 Process Flow

The following describes the end-to-end process flow of the application from user input to final output:

Step	Action	Description
1	User Input	User enters a manufacturing concept in the text field or selects from the example dropdown in the Streamlit UI
2	API Key Validation	The application checks that a Gemini API key has been entered in the sidebar. If absent, the Generate button remains disabled.
3	Generate Triggered	User clicks the Generate button. The application captures the concept text and selected visual style.
4	LLM API Call	The Gemini 2.5 Flash model is invoked with a structured prompt requesting a five-section technical narrative about the concept.
5	Image API Call	A Pollinations.ai URL is constructed by encoding the concept and style into query parameters. An HTTP GET request fetches the generated image.
6	Render Outputs	The text narrative is rendered as formatted markdown in the left column. The generated image is decoded from bytes and displayed in the right column.
7	Workflow Display	Below the outputs, a static workflow diagram illustrates the multimodal pipeline from input to both outputs.

2.3 Information Flow

The information flow describes how data moves through the system from the point of user input to the final rendered outputs.

Outbound- Text Generation:

- The user's concept string is embedded into a structured prompt template
- The prompt is sent to the Gemini API endpoint via the google-generativeai SDK
- The API returns a text response object containing the generated narrative
- The response text is extracted and rendered as markdown in the Streamlit UI

Outbound- Image Generation:

- The user's concept string and selected visual style are concatenated into an image prompt string
- The string is URL-encoded using Python's urllib.parse.quote function
- An HTTP GET request is sent to the Pollinations.ai endpoint with the encoded prompt as a path parameter
- The response body contains raw image bytes in JPEG or PNG format

- The bytes are decoded using Pillow (PIL) and rendered via st.image in Streamlit

No data is stored, logged, or persisted at any point. The Gemini API key is held only in the Streamlit session state for the duration of the user session.

2.4 Components Design

The system is composed of the following key components:

Component	Technology	Responsibility
Streamlit UI	Streamlit (Python)	Renders the user interface including input fields, sidebar settings, output columns, and workflow diagram
Input Handler	Python	Captures concept text from text input or dropdown selection and passes it to both API calls
LLM Orchestrator	google-generativeai SDK	Configures the Gemini client with the user API key, constructs the structured prompt, calls the model, and returns the response text
Image Orchestrator	requests + Pillow	Constructs the Pollinations.ai URL with encoded parameters, performs the HTTP GET, decodes the image bytes, and passes the image to Streamlit for display
Workflow Visualizer	Streamlit native	Renders the static multimodal pipeline diagram below the outputs using Streamlit columns and metric-style boxes

2.5 Key Design Considerations

Stateless Architecture

The application maintains no server-side state and uses no database. Each session is independent. The Gemini API key is stored only in Streamlit session memory and is never written to disk or logs. This keeps the application lightweight and eliminates data persistence concerns.

No API Key for Image Generation

Pollinations.ai provides free, open access to the Flux diffusion model without requiring authentication. The image is generated by encoding the prompt directly into the request URL. This eliminates the need for a second API key and simplifies the user experience.

Parallel Model Invocation

Both the text and image generation calls are triggered from the same user prompt within a single button click. While Streamlit executes them sequentially due to its synchronous execution model, they are logically parallel neither model depends on the output of the other. This design demonstrates true multimodal generation from a shared input.

Prompt Engineering for Manufacturing Domain

The LLM prompt is engineered to produce a consistent five-section output: Overview, Working Principle, Key Components, Manufacturing Application, and Advantages. This structured format ensures the output is professional, educational, and directly useful for presentations. The image prompt is augmented with style descriptors and domain keywords to improve visual quality and relevance.

Error Handling

Both API calls are wrapped in try-except blocks. If the LLM call fails, an error message is displayed in the text column without affecting the image generation, and vice versa. This prevents a failure in one model from blocking the other output.

2.6 API Catalogue

API	Type	Auth	Purpose
Google Gemini API	REST / SDK	API Key	Invokes Gemini 2.5 Flash to generate structured technical narratives from manufacturing concept prompts
Pollinations.ai (Flux)	HTTP GET	None (free)	Generates engineering illustrations and visual prototypes from text prompts encoded in the request URL

Google Gemini API endpoint:
https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent

Pollinations.ai image endpoint pattern:
https://image.pollinations.ai/prompt/{encoded_prompt}?width=800&height=600&model=flux&nologo=true&seed=42

3. References

- **Google Gemini API Documentation:** https://ai.google.dev/gemini-api/docs
- **Pollinations.ai:** https://pollinations.ai
- **Streamlit Documentation:** https://docs.streamlit.io
- **Pillow (PIL) Library:** https://pillow.readthedocs.io
- **Flux Image Model:** https://blackforestlabs.ai
- **Python urllib.parse:** https://docs.python.org/3/library/urllib.parse.html
- **google-generativeai Python SDK:** https://pypi.org/project/google-generativeai

4. GitHub

The complete source code for the Multimodal Manufacturing Creator is available on GitHub at the following repository:

https://github.com/omtiwari17/Multimodal-Manufacturing-Creator

The repository contains the following files:

File	Description
app.py	Main Streamlit application contains all UI, LLM orchestration, image generation, and workflow display logic
requirements.txt	Python dependencies required to run the application
README.md	Project overview, setup instructions, tech stack, and example concepts
.gitignore	Excludes Python cache, environment files, and Streamlit secrets from version control